데이터베이스 · 저장소 설계 문서

CopyLane · 최종 산출물 (2차 제출본 개정)

작성 **권소라 (원 작성자)**
개정 **2026-10-21 · 기준 커밋 ksr b54246d · 10-22 확인 반영**
스키마 **31표 · 뷰 5 · alembic head 0027**
실측 **이 PC DB · 2026-10-21 15:44 KST · SELECT 만**

SKN Final Project · 3팀 「한끗」

목차

1. 한 장 요약 4

1.1 이 문서가 답하는 지적 4

1.2 왜 별도 벡터DB 를 쓰지 않는가 (D-41) 5

2. 층 구조 — 왜 두 층인가 6

3. 설계 원칙 — 전부 결정에 근거가 있다 7

4. 개체 관계 (ERD) 8

4.1 외래키 32개 — 2026-10-21 이 PC DB 9

5. 데이터 층 명세 19표 · 정본 db/schema.sql 11

5.1 거버넌스 (5) 11

5.2 원문 · 문장 (2) 11

5.3 RAG (2) 11

5.4 위험도 — RAG 가 아니라 관계형 (3) 12

5.5 사전 · 제품 사실 (2) 12

5.6 골든셋 · 학습 (3) 12

5.7 세그먼트 · 재현성 (2) 13

6. 런타임 층 명세 12표 · 정본 app/models.py 14

6.1 품목(product category)이 문서가 아니라 판정에 있는 이유 15

6.2 판정 표 — 판정 결과의 네 값과 위험도 세 칸 15

6.3 검수 이력 저장 — 2차 이후 추가 16

7. 인증 18

7.1 왜 계정 테이블이 거버넌스의 일부인가 18

7.2 설계 — 계정 표가 둘이다 18

7.3 일반 회원 테이블 — 2차 때는 없었고 0017 에서 생겼다 18

8. 저장소(파일시스템) 설계 — 위치가 곧 권한 19

8.1 기기 사이의 공유 — 2차 이후 추가 19

9. 뷰 — 실수를 구조로 막는다 21

10. 인덱스 전략 22

11. 마이그레이션 이력 23

12. 실측 검증 25

12.1 스키마 객체가 설계대로 있다 — 2026-10-21 15:44 KST · 이 PC · SELECT 만 25

12.2 제약 8종이 위반을 거부했다 — 2026-09-17 실측 25

12.3 캐스케이드 삭제로 9개 테이블이 전부 0 이 됐다 — 2026-09-17 실측 26
12.4 적재 현황 — 이 PC DB · 2026-10-21 15:44 KST 26
13. 미확정 · 검토 중 (2026-10-21) **28**
13.1 근거 신뢰도 등급(evidence_tier) 신설 제안 — 승인 전 28
14. 근거 **30**

1. 한 장 요약

> **작성** 권소라(원 작성자) · **개정** 2026-10-21 · **기준 커밋** `ksr b54246d` (1차 작성은 `f8ebb09`) · **판** v3.0(최종 · 2026-10-22 확인 반영) **대상** 공식 WBS 「데이터 수집·저장」 단계 산출물 · 2차 제출본(2026-09-17, v2.0)을 2026-10-21 저장소 상태로 고쳤다 **2차 대비** 런타임 층 7표 → 12표(회원 · 공지 · 약관 · 문의 · 오류 로그) · 마이그레이션 0015 → 0027 · 검수 이력 저장 · 판정 표의 인코더 하한 · 파일 저장소의 공유 구조 · ERD 를 현재 표 기준으로 다시 그림 **수치 출처** 스키마 값은 이 PC 의 PostgreSQL(127.0.0.1:5432 · 개발용 복제본 `DATA_ROLE=replica`)을 **2026-10-21 15:44 KST 에 SELECT 로만** 조회한 값이다. 이 DB 의 alembic 판은 저장소 head 와 같은 `0027_judgment_encoder_basis` 다. 행 수는 같은 조회값이고, 정본 기기(클론 B)의 수와 다를 수 있다.

항목	2026-10-21 현재	근거
스택	PostgreSQL 16.15 + pgvector 0.8.6 · SQLAlchemy 2.0 + Alembic	이 PC DB `version()` · `pg_extension`
층 구성	거버넌스·데이터 층 **19표** + 런타임 층 **12표** = **31표**(+ `alembic_version`) · 뷰 **5** · 열거형 **14종**	이 PC DB `pg_tables` · `pg_views` · `pg_enum`
무결성	CHECK **53**(데이터 층 17 · 런타임 층 36 · 이름 없는 CHECK 2 포함 — `ck_` 이름만 세면 51) · 외래키 **32**(CASCADE 25 · SET NULL 2 · NO ACTION 5) · 유일 제약 4	이 PC DB `pg_constraint`
인덱스	**70**(기본키 · 유일 인덱스 포함 · GIN 2 · 부분 인덱스 5) · 벡터 인덱스 없음	이 PC DB `pg_indexes`
마이그레이션	Alembic 단일 체인 **0001 ~ 0027**(파일 27개) · head **`0027_judgment_encoder_basis`**	`alembic/versions/` · 이 PC DB `alembic_version`
설계의 축	모든 파생물이 `fragment_id` 하나에 매달린다 — 소스를 지우면 문장·청크·벡터·사전·골든셋이 함께 사라진다	`db/schema.sql` · D-20
저장소의 축	디렉터리가 곧 권한이다 — 이용조건 등급별 물리 분리 · `data/**` 는 커밋하지 않는다 · 파생물은 원장(git)과 팀 공유 저장소로 나른다	`.gitignore` · D-19 · D-247 · D-249
선언과 실물 일치	런타임 층 12표의 칸 · CHECK 이름이 `app/models.py` 와 DB 에서 하나도 다르지 않다. 거버넌스 층의 제약 · 인덱스 · 뷰 · 열거형 이름이 `db/schema.sql` 과 DB 에서 같다	`app.models` 메타데이터와 DB 카탈로그를 직접 대조(2026-10-21)

1.1 이 문서가 답하는 지적

중간발표(2026-09-15) · 멘토링(2026-09-16) · 강사 가이드(2026-08-28)에서 나온 지적을 「현 진행 → 지적 → 협의 → 결론」 네 칸으로 해당 절에 붙였다. 2026-10-21 개정에서는 「현 진행」 칸을 지금 값으로 고치고, 보류 비율 이슈(§6.3)의 「결론」 칸에 최신 측정 한 문장을 더했다. 중간발표 지적은 원 기록(중간발표 Q&A 정리)과 다시 대조해 고쳤다. 멘토링 · 강사 가이드는 원 기록을 찾지 못해 2차 제출본의 문장을 옮겼고, 그 칸에 「2차 제출본 기록」이라고 밝혔다.

지적	출처	위치
별도 벡터DB 대신 pgvector — 데이터가 늘면 한계는 없는가	중간발표 09-15	§1.2
캐스케이드가 안 걸린 데이터가 있었다 — pgvector 문제라기보다 관계 설계 문제	중간발표 09-15	§4
보류가 많으면 서비스가 한 단계 더 된다 — 보류 비율을 낮추는 것이 중요	중간발표 09-15	§6.3

지적	출처	위치
결과를 확정적 판정처럼 제시하면 안 된다	강사 가이드 08-28(2차 제출본 기록)	§ 6.3

1.2 왜 별도 벡터DB 를 쓰지 않는가 (D-41)

소스 회수 요청에 한 줄로 응할 수 있어야 하기 때문이다. 벡터를 다른 인스턴스에 두면 `DELETE FROM source` 가 벡터까지 지우지 못하고, 그 순간 「회수했다」는 말이 거짓이 된다. 데이터가 쌓인 뒤에는 이 경로를 만들 수 없다(D-20).

① 기존 PostgreSQL 인프라를 그대로 쓴다 ② 메타데이터와 벡터를 한 곳에서 관리한다 ③ 원본–파생 관계와 삭제를 같은 트랜잭션에 건다 ④ 이 규모에서 운영 복잡도가 줄어든다.

제기된 이슈 — 사용자·데이터가 늘면 pgvector 로 감당되는가

구분	내용
현 진행 (2026-10-21)	PostgreSQL 16 + pgvector 단일 인스턴스. 청크 **6,686** · 임베딩 **6,686**(KURE-v1 · 1024차원) 적재(이 PC DB · 원장 10-13(B)). 벡터 인덱스(HNSW · IVFFlat)는 여전히 만들지 않았다(§ 10)
지적 중간발표 09-15	질문 — pgvector 를 쓰면서 생긴 문제나 한계는 없는가. 피드백 — 프로젝트 기간과 데이터 규모를 보면 별도 Vector DB 없이 PostgreSQL 과 pgvector 를 통합한 선택은 합리적이다. 다만 데이터가 대규모로 늘면 전문 Vector DB 대비 검색 성능 · 확장성에서 한계가 생길 수 있다
협의 멘토링 09-16(2차 제출본 기록)	수십만 건 수준에서는 과부하가 걸리지 않는다. 지금 단계에서 고려할 문제가 아니고, 대비책은 테이블을 잘 쪼개고 FK 로 잇는 것이다
결론	pgvector 단일 인스턴스로 그대로 간다. 확장 대비 별도 작업은 하지 않고 테이블 분할 + FK 구조(§ 4)를 유지한다. 벡터 인덱스는 청크가 만 단위가 되면 실측해서 만든다(D-192). 데이터가 대규모로 늘면 전문 Vector DB 분리를 다시 검토한다

> 한계를 지우지 않는다. 이 선택의 대가는 둘이다 — ① 대규모에서 전문 Vector DB 대비 검색 성능이 떨어질 수 있다 ② 관계 설계가 정확해야 캐스케이드가 동작한다(§ 4). 둘 다 지금 규모에서는 비용이 아니고, 회수 경로(D-20)를 얻는 대가로 받아들였다. 참고로 판정 요청 1번의 응답 시간은 542 ms 이고 그중 근거 검색(retrieve) 노드가 291 ms 다(원장 10-20 ① · 예열 뒤 3번 중앙값).

2. 층 구조 — 왜 두 층인가

	거버넌스 · 데이터 층 19표	런타임 층 12표
무엇	우리가 수집한 것 — 법령 · 의결서 · 사전 · 제재표 · 골든셋	사용자가 만든 것 — 문구 · 판정 · 계정 · 문의 · 운영 기록
정본	`db/schema.sql` (동결본 `db/schema_0001.sql` · D-225)	`app/models.py` (ORM · D-89)
마이그레이션	`db/migrations/*.sql` 본문을 Alembic 이 읽어 실행	Alembic 파일에 DDL 을 직접 적는다
문서 개체	`document` · `sentence`	`work_doc` · `copy_sentence`
출처	수집 (`fragment` 유래)	사용자 입력 · 관리자 입력
회수 경로	`source` → `fragment` → 파생물 CASCADE	`work_doc` → 하위 5표 CASCADE

두 층을 같은 이름으로 부르지 않는다. 데이터 층의 `document` · `sentence` 는 수집한 원문(`fragment_id` FK)이다. 사용자가 쓴 문구는 fragment 에서 오지 않으므로 다른 개체이고, 이름을 `work_doc` · `copy_sentence` 로 갈랐다. 같은 이름을 쓰면 조인 한 번에 층이 섞인다. **두 층 사이의 FK 는 0개다**(§ 4 의 FK 표 — 런타임 표는 `work_doc` · `copy_sentence` · 두 계정 표에만 매달린다).

`db/migrations/` 폴더에 번호가 빠진 곳(0012 · 0014 · 0016 ~ 0018 · 0022 · 0026 · 0027)이 있는 것은 이 때문이다 — 런타임 층을 고치는 판은 SQL 파일 없이 Alembic 파일에 DDL 을 적는다(각 파일 머리말).

3. 설계 원칙 — 전부 결정에 근거가 있다

#	원칙	스키마에서 어떻게 구현됐나	근거
1	등급은 소스가 아니라 추출물의 속성	`fragment.grade` 가 독립 칸 — 소스 등급을 상속하지 않는다	D-18
2	소스 회수 요청에 응할 수 있어야 한다	모든 파생 테이블에 `fragment_id … ON DELETE CASCADE`	D-20
3	벡터도 함께 사라져야 한다	`chunk_embedding` 을 같은 인스턴스에 — `vector(1024)`	D-41
4	2인 확인은 문서가 아니라 제약	`CHECK (grade_decided_by <> grade_reviewed_by)` · 제재표는 `v_risk_lookup` 이 2인 서명 행만 낸다	D-66
5	위험도는 블랙박스 점수가 아니다	[별표]를 관계형 테이블로 — 코드가 직접 조회	기획 3-4
6	데이터 재배포와 모델 배포는 다른 축	`provenance` · `redistributable` 분리	D-71
7	시정이 의도대로 됐는지 잰다	`certainty` · `tense` · `sent_type` 3축	D-74
8	정규화는 비파괴	`raw` / `norm` / `offset_map` 3열 분리(데이터 층 `sentence` · 런타임 층 `copy_sentence`)	D-84
9	최종 위험도는 하한 아래로 내려가지 않는다(래칫)	`judgment` 의 `ck_judgment_final_not_below_floor` · `ck_judgment_final_not_below_encoder`	D-09 · 0022 · 0026
10	저장된 등급마다 근거가 남는다	하한 위로 올리면 `evidence_span` 필수 · 인코더 하한에는 `evidence.encoder_refs` 필수	D-224 · 0026 · 0027

9 · 10 은 2차 제출(09-17) 뒤에 DB 제약으로 더해진 원칙이다. 판정 계약(`app/contracts.py`)이 거부하는 값을 DB 도 똑같이 거부하게 맞췄다(D-99 — 같은 규칙을 두 곳이 다르게 들지 않는다).

4. 개체 관계 (ERD)

아래 두 그림은 2026-10-21 이 PC DB 의 카탈로그(`information_schema` · `pg_constraint`)에서 표 · 기본키 · 외래키 · 주요 칸 · 제약을 옮겨 다시 그린 것이다. 칸은 전부 싣지 않고 관계와 규칙을 읽는 데 필요한 것만 실었다(전체 칸은 §5 · §6 과 `db/schema.sql` · `app/models.py`).

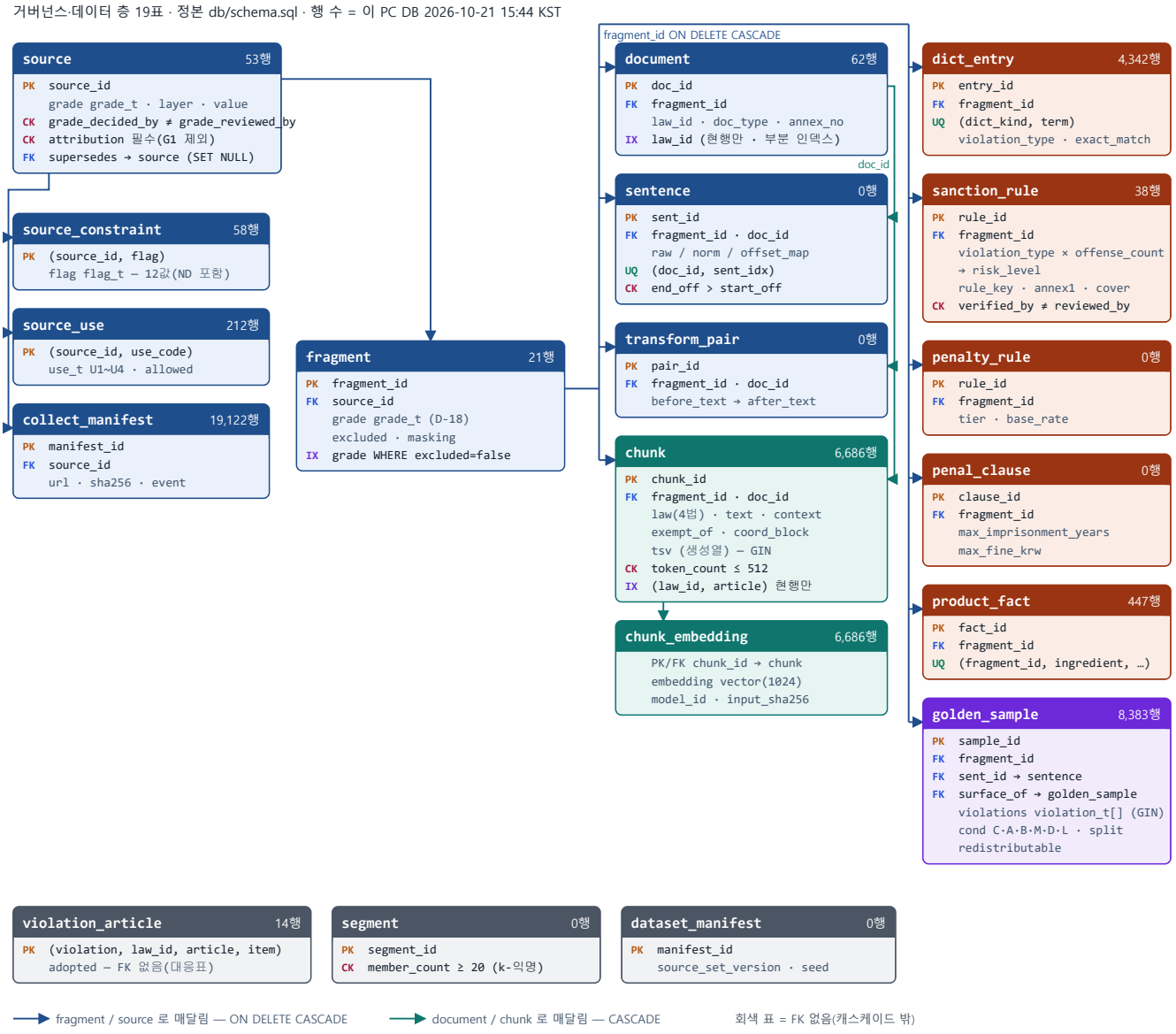


그림 1 — 파란 선은 전부 `ON DELETE CASCADE` 다. `source` 하나를 지우면 `fragment` 와 그 아래 파생물이 통째로 사라진다. 청록 선은 `document` · `chunk` 에 매달린 2단 CASCADE 다. `golden_sample` 은 `sentence` (sent_id)와 자기 자신(surface_of · 표면 변형의 원문)에도 CASCADE 로 매달린다. 회색 세 표(`violation_article` · `segment` · `dataset_manifest`)는 FK 가 없어 캐스케이드 밖이다.

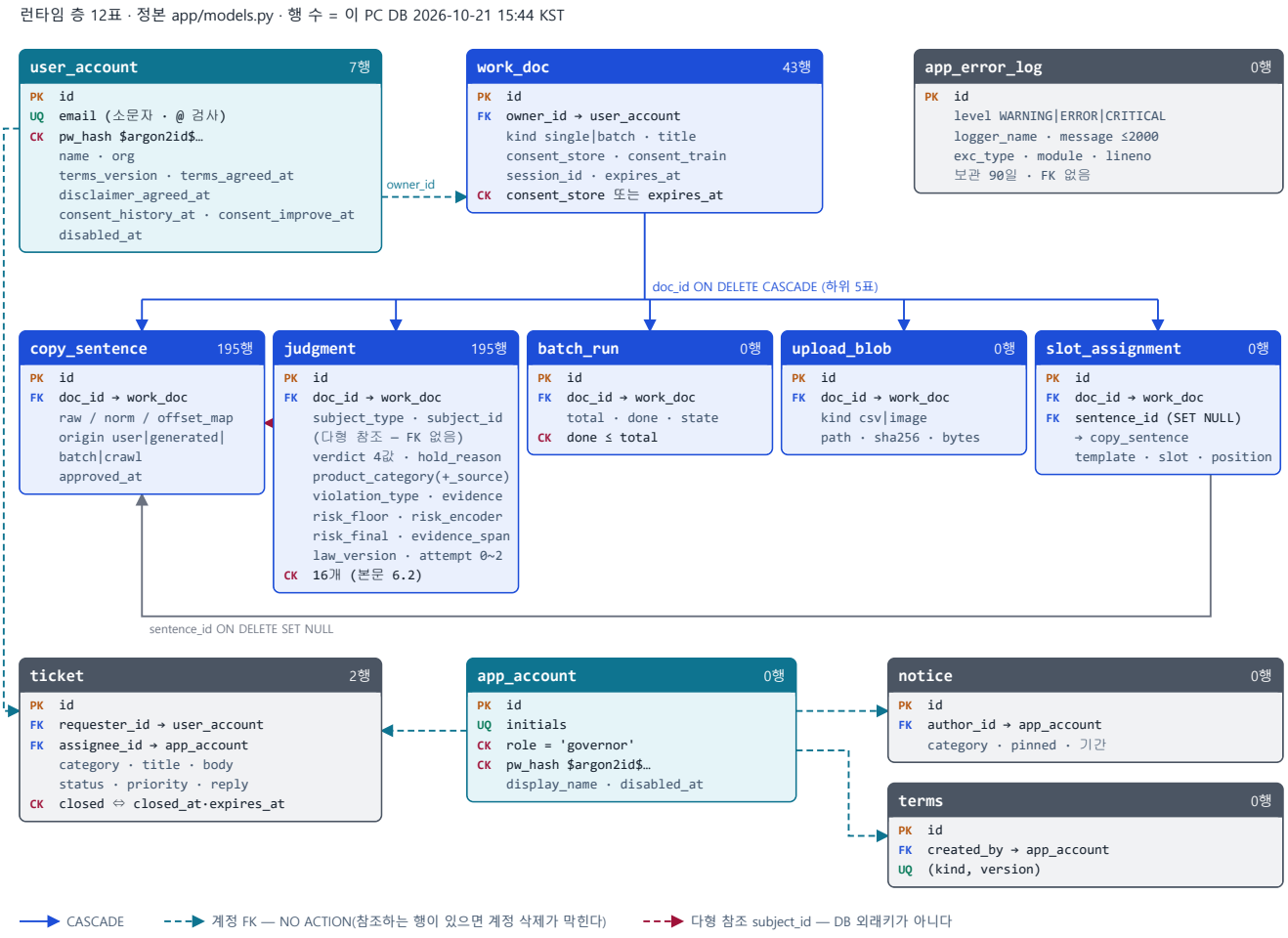


그림 2 — `work_doc` 이 하위 5표의 CASCADE 루트다. `judgment` 는 다형 참조(`subject_type` · `subject_id`)라 문장에 대한 DB 외래키가 없고, 대신 `doc_id` FK 로 문서가 지워질 때 함께 지워진다. 계정 표(`user_account` · `app_account`)를 가리키는 FK 다섯은 `NO ACTION` 이다 — 참조하는 행이 있으면 계정 행을 지울 수 없다. 사용자 탈퇴는 행 삭제가 아니라 `user_account.disabled_at` 기록이다(`app/routers/user.py` 「끄기」).

4.1 외래키 32개 — 2026-10-21 이 PC DB

층	부모 표	자식 (칸)	삭제 규칙
데이터	`source`	`source_constraint` · `source_use` · `collect_manifest` · `fragment` (source_id)	CASCADE 4
데이터	`source`	`source` (supersedes — 판 계보)	SET NULL
데이터	`fragment`	`document` · `sentence` · `chunk` · `dict_entry` · `sanction_rule` · `penalty_rule` · `penal_clause` · `product_fact` · `golden_sample` · `transform_pair` (fragment_id)	CASCADE 10
데이터	`document`	`sentence` · `chunk` · `transform_pair` (doc_id)	CASCADE 3
데이터	`chunk` · `sentence` · `golden_sample`	`chunk_embedding` (chunk_id) · `golden_sample` (sent_id) · `golden_sample` (surface_of)	CASCADE 3
런타임	`work_doc`	`copy_sentence` · `judgment` · `batch_run` · `upload_blob` · `slot_assignment` (doc_id)	CASCADE 5

층	부모 표	자식 (칸)	삭제 규칙
런타임	`copy_sentence`	`slot_assignment` (sentence_id)	SET NULL
런타임	`user_account`	`work_doc` (owner_id) · `ticket` (requester_id)	NO ACTION 2
런타임	`app_account`	`notice` (author_id) · `terms` (created_by) · `ticket` (assignee_id)	NO ACTION 3

합계 — 데이터 층 21(CASCADE 20 · SET NULL 1) · 런타임 층 11(CASCADE 5 · SET NULL 1 · NO ACTION 5). 2차 제출본의 27개에서 0017 이 더한 계정 FK 5개가 늘었다.

제기된 이슈 — 캐스케이드가 적용되지 않는 데이터가 있었다

구분	내용
현 진행 (2026-10-21)	데이터 층의 세로 연결은 전부 `ON DELETE CASCADE` 다(위 표 20개). 2026-09-17 실측에서 `DELETE FROM source` 한 줄로 9개 표가 전부 0 이 됐다(§12.3). 그 뒤 데이터 층에 FK 는 새로 생기지 않았다
지적 중간발표 09-15	팀이 답변에서 「시험 과정에서 부모–자식 관계가 명확히 정의되지 않은 데이터는 캐스케이드 삭제가 적용되지 않았다」고 밝혔고, 이에 대한 피드백은 「캐스케이드 문제는 pgvector 자체의 문제라기보다 관계형 DB 의 데이터 관계 설계 문제에 가깝다」였다
협의 멘토링 09-16(2차 제출본 기록)	대비책은 테이블을 잘 쪼개고 FK 로 잇는 것이다. 팀은 이를 관계를 명시적으로 정의하는 방향으로 받아들였다
결론	캐스케이드가 닿는 범위는 관계를 어떻게 정의했느냐가 결정한다. 세로로 이어지지 않은 표는 삭제가 전파되지 않으며, 그것은 저장소의 한계가 아니라 그 표를 파생물로 두지 않았다는 뜻이다. 표를 추가할 때 `fragment_id` 종속 여부를 먼저 정한다. 집계·메타 표(`segment` · `dataset_manifest`)의 회수 절차는 §13 의 남은 항목이다

5. 데이터 층 명세 19표 · 정본 `db/schema.sql`

행 수는 2026-10-21 15:44 KST 이 PC DB 조회값이다. 2차 제출(09-17) 뒤 바뀐 칸에는 마이그레이션 번호를 붙였다.

5.1 거버넌스 (5)

테이블	행	역할	핵심 제약
`source`	53	소스 레지스트리 — `data_sources.yaml` 의 DB 표현	`ck_source_four_eyes` 판정자≠확인자 · `ck_source_by_attr` G1 이 아니면 attribution 필수
`source_constraint`	58	제약 플래그 (1:N)	`flag_t` **12종** — NOREDIST · NOSTORE · QUERYLOG · PREAPPROVAL · NOTRAIN · **ND(변경금지 · 0020)** 포함
`source_use`	212	U1~U4 허용 여부	`registry.require()` 가 참조하는 실행 게이트
`collect_manifest`	19,122	수집 이력 · 재현성	같은 sha256 은 다시 넣지 않는다 · DB 의 `source` 에 있는 원천의 줄만 넣는다(`scripts/load_db.py` `load_manifest`)
`fragment`	21	추출 단위 — 등급이 갈리는 지점	`grade` 독립 칸(D-18) · `excluded` · 부분 인덱스 `ix_fragment_grade`

- `source` 53 은 레지스트리 등재 70 중 적재된 몫이다. 나머지 17 은 2인 확인 미완 · attribution 없음으로 CHECK 를 지나지 못해 넣지 않고 이름을 낸다(원장 10-06 ① · D-66).
- `collect_manifest` 19,122 행과 수집 원장 파일 `data/manifest.jsonl` 28,277 줄(`ksr b54246d`)은 같은 수가 아니다. 적재 코드는 `source` 에 없는 원천의 줄과 이미 들어간 sha256 을 넣지 않는다. 그 차이를 원천별로 나눠 세지는 않았다(§ 13).

5.2 원문 · 문장 (2)

테이블	행	역할 · 주의
`document`	62	문서 단위(법령 · 행정규칙 · 별표) · `superseded_at IS NULL` 이 현행 · `annex_no` (0015 · 별표를 조문과 다른 문서로)
`sentence`	0	`raw` (표시) / `norm` (매칭) / `offset_map` (대응) 3열 · `ck_sentence_span` · `uq_sentence_pos` . 현재 적재기 (`scripts/load_db.py`)에 이 표를 채우는 단계가 없어 비어 있다

5.3 RAG (2)

테이블	행	역할 · 주의
`chunk`	6,686	조·항·호 단위 청크 · `ck_chunk_tokens` token_count ≤ 512 · `tsv` 생성열 + GIN(0010 · 어휘 검색) · `context` (0008 · 검색이 보는 문맥 — 인용은 `text`) · `part_no` / `part_total` (0011 · 쪼갠 조각) · `input_token_count` (0011) · **`law`** (0019 · 4법 CHECK — 표시광고법 · 식품표시광고법 · 화장품법 · 건강기능식품법) · **`exempt_of`** (0021 · 적용 제외 목의 부모 경로 · 별표만) · **`coord_block`** (0025 · 인용 좌표를 막는 이유 — 빈 문자열 · 겹침 · 순서)
`chunk_embedding`	6,686	`vector(1024)` · `model_id` = `nlpai-lab/KURE-v1` (6,686 전부) · `input_sha256` (0008 · 벡터를 만든 입력 문자열의 지문)

- 법별 청크(이 PC DB): 화장품법 3,483 · 식품표시광고법 2,091 · 표시광고법 589 · 건강기능식품법 523. 건강기능식품법은 참고 전용이다(D-271 ⑥). 판정 그래프에 이 법의 노드가 없어(`app/graph.py` `LAW_OF_NODE` 는 세 법뿐) 위반 근거로도, 판정 뒤 `references` 가 고르는 화면 참고 조문으로도 쓰이지 않는다. 이 법 청크가 결과로 나오는 곳은 조문 검색 API `POST /search` 뿐이다.
- `input_token_count` 가 512 를 넘는 청크 3개 · 최댓값 564. 이 칸에는 상한 CHECK 를 걸지 않았다 — 리랭커가 미선정이라 지킬 상한이 없다(D-200).
- `exempt_of` · `coord_block` 의 NULL 은 「아직 재적재 안 됨」 이다. 빈 문자열(「해당 없음」)로 미리 채우지 않는다 — 채우면 재적재 전에 겹친 경로에 인용 좌표가 선다(0021 · 0025 머리말).

5.4 위험도 — RAG 가 아니라 관계형 (3)

테이블	행	담는 것
`sanction_rule`	38	행정처분 기준 [별표] — 위반유형 × 차수 → 처분종류·값 · 위험도. 0023 이 서명된 원천 (`scripts/sanction_review.yaml`)의 칸 **`rule_key`** · **`annex1`** · **`cover`** · **`quote`** · **`fact_kind`** · **`plan_sha`** 와 `ck_sanction_cover` · 부분 유일 인덱스 `ux_sanction_rule_key` 를 더했다(D-305 · D-309 · D-310) · 조회 경로 `ix_sanction_lookup`
`penalty_rule`	0	과징금 고시 — 부과기준율·정액·가중·감경. 적재기 없음
`penal_clause`	0	벌칙 — 징역·벌금 상한(R4 근거). 적재기 없음

왜 벡터가 아닌가. 별표를 평문화해 임베딩하면 ① 표 구조가 깨져 「차수」와 「일수」의 대응이 사라지고 ② 검색이 확률적이라 같은 입력에 다른 위험도가 나올 수 있으며 ③ 「블랙박스 점수는 검수 도구로서 실격」이라는 전제에 걸린다. 실제 값은 `v_risk_lookup` 뷰를 코드가 조회한다. 이 뷰는 2인 서명(`verified_by` · `reviewed_by`)이 끝난 행만 낸다 — 이 PC DB 에서 38 행 전부가 보인다.

5.5 사전 · 제품 사실 (2)

테이블	행	역할	주의
`dict_entry`	4,342	금지·적법·질병·완화금지 사전(`dict_kind` · 설계값 넷). 이 PC DB 의 4,342 행은 전부 `금지표현` 이다	`exact_match` — 정확매칭만 위험도 하한 자격 · `uq_dict_term` . 사전 판 `2ff0168c4a28` · 4,342 종(원장 10-19 ①)
`product_fact`	447	인정 기능성 문구	A 자격형 판정(D-59)의 근거 · `uq_product_fact`

5.6 골든셋 · 학습 (3)

`golden_sample` 이 이 스키마에서 열이 가장 많다(24칸) — 라벨 축 · 측정 축 · 출처 추적 · 조건.

축	컬럼	근거
위법 유형 (다중)	`violations violation_t[]` — 11종 · GIN 인덱스	D-178
3요소 (허위·오인·비방)	`three_elem JSONB`	—
불가 사유	`infeasibility` A / B / C	D-59

축	컬럼	근거
위험도 순서형	`risk` R0 ~ R4	—
측정 축	`certainty` · `tense` · `sent_type`	D-74
주장 스팬	`claim_spans JSONB`	D-30
근거 좌표	`evidence JSONB` `[{law_id, article, item}]` · **`evidence_candidate`** (0024 · 후보 근거)	D-282 · D-317
조건	**`cond`** (0024) — C 절대형 · A 자격형 · B 실증형 · M 모호 · D 판정 대상 아님 · L 적법 · 조건 없는 원천은 NULL	D-317
출처 추적	`provenance` · `redistributable`	D-71
분할	`split` train / dev / test_sentence / test_holdout	D-171
표면 변형	`surface_variant` S0~S5 · `surface_of` — `ck_surface_pairing`	D-91

- 적재된 판: 골든 `94a79fe34363` · **8,383 행**(train 5,894 · 평가 분할 test_sentence 2,489) — 이 PC DB · 원장 10-18 ⑭. 2,489 는 골든의 평가 분할이고, 판정 그래프의 봉인 시험지는 10-15 재분할(D-347) 뒤 그중 1,141 행(지문 `74e37657d39ec325`)으로 아직 열지 않았다. 옛 2,489 는 10-14 인코더 단독으로 한 번 쟀다(보고 전용). dev 는 DB 의 분할 값이 아니라 train 에서 떼어 낸 파일로 쓴다(원장 · 749 행).
- 조건별(이 PC DB): B 2,764 · C 1,584 · 없음 1,111 · A 973 · D 960 · M 913 · L 78.
- 0024 의 두 제약 — `ck_golden_cond_empty` (D · L 행은 위반 · 근거가 비어야 한다) · `ck_golden_cond_basis` (C · A · B 행은 근거나 후보 근거가 있어야 한다). `violations` 가 비었다고 적법이 아니므로 골든을 DB 에서 읽을 때는 뷰 `v_golden_scored` · `v_golden_legal` 을 지난다(§9).
- `ck_golden_injected_not_holdout` — 주입본이 평가 split(`test_sentence` · `test_holdout`)에 들어가는 것을 DB 가 거부한다. 2026-09-10 정정 뒤의 정의가 그대로다.

테이블	행	역할
`transform_pair`	0	시정 전후 문구 쌍(`document` 에도 CASCADE). 적재기 없음
`violation_article`	14	위반 유형 → 조문 대응표. 타입이 아니라 데이터다(표 주석) · FK 없음

5.7 세그먼트 · 재현성 (2)

테이블	행	핵심
`segment`	0	`ck_segment_k_anon` member_count ≥ 20 — k-익명성 미달 군집은 생성 자체가 차단
`dataset_manifest`	0	`source_set_version` 이 바뀌면 재학습 트리거 · `unmeasurable[]` 에 30건 미만 유형 기록

두 표 모두 표만 있고 채우는 코드가 없다. 세그먼트·페르소나는 설계 단계이고 결론 재검토 대기다(D-270).

---

6. 런타임 층 명세 12표 · 정본 `app/models.py`

`db/schema.sql` 은 이 층을 담지 않는다(D-89). 2차 제출 때 7표였고 0016 · 0017 이 다섯 표를 더했다. 「쓰는 코드」 칸은 2026-10-21 `ksr b54246d` 의 `app/` · `scripts/` 에서 표 이름 · 모델 이름으로 찾은 결과다.

테이블	행	역할	핵심	쓰는 코드
`work_doc`	43	사용자 작업 문서 — 문구 하나 = 문서 하나	`kind` single / batch · `owner_id` → `user_account` (0017) · `consent_store` · `consent_train` · 하위 5표의 CASCADE 루트 · `ck_work_doc_expiry_without_consent`	`app/judgment_store.py`
`copy_sentence`	195	사용자가 쓰거나 생성한 문구 한 줄	`raw` / `norm` / `offset_map` — 데이터 층과 같은 비파괴 계약 · `origin` user / generated / batch / crawl	`app/judgment_store.py`
`judgment`	195	판정 — 다형 참조	§6.2 · CHECK **16개**로 이 스키마에서 가장 많다	`app/judgment_store.py`
`batch_run`	0	배치 검수 1회 (CSV 일괄)	`ck_batch_run_progress` done ≤ total	없음(표만)
`upload_blob`	0	사용자 업로드물(CSV · 이미지)	경로가 여기에만 있다 · 학습 금지(NOTRAIN)가 기본	없음(표만)
`slot_assignment`	0	템플릿 조립 (AI 광고 생성)	승인 객체만 받는다 · `sentence_id` SET NULL	없음(표만)
`app_account`	0	관리자 콘솔 계정 (`governor`)	§7 · 가입 화면 없음	`scripts/admin_account.py` (런처 메뉴 35) · `app/routers/auth.py`
`app_error_log`	0	WARNING 이상 앱 오류 로그(0016 · D-261)	마스킹 필터를 지난 문장과 예외 클래스 이름만 · 트레이스백 · 요청 경로 · IP 칸이 없다 · `message` ≤ 2,000자 · 보관 90일	`app/error_log.py` (백그라운드 기록 · 기한 지난 행 삭제)
`user_account`	7	일반 회원 (0017 · D-260)	§7.3	`app/routers/user.py` (가입 · 마이페이지 · 끄기)
`notice`	0	공지	`category` rule_change / system / update · 게시 기간 CHECK	`app/routers/admin_board.py` (작성 · 수정)
`terms`	0	약관 판	`kind` service / privacy / disclaimer · `uq_terms_kind_version`	넣는 코드 없음(`admin_terms.py` 는 읽기만)
`ticket`	2	문의	판정 원문 · 판정 id 를 담는 칸이 없다(D-76) · 닫히면 `closed_at` 과 파기 기한 `expires_at` 을 같이 채운다(`ck_ticket_closed_expiry`)	`app/routers/user.py` (접수) · `app/routers/admin_cs.py` (처리 · 완료 시 `expires_at` = `closed_at` + 180일)

이 표의 행 수는 개발용 복제본 PC 의 시험 사용분이다. 서비스 이용 수치로 읽지 않는다.

6.1 품목(product category)이 문서가 아니라 판정에 있는 이유

배치 검수는 한 실행 안에서 SKU 마다 카테고리가 다르고(건기식·화장품 혼재), 카테고리는 우리가 판별한다(D-82). 따라서 카테고리는 문서의 속성이 아니라 판정의 결과다. 0018 이 출처 칸 `product_category_source`(classified 판별 · user_selected 사용자가 고름)를 더했고, 품목이 있으면 출처도 있어야 한다(`ck_judgment_category_source_pair` · D-276).

6.2 판정 표 — 판정 결과의 네 값과 위험도 세 칸

값	뜻	규칙
`confirmed`	확정 — 판정 절차 종료	위반이 없으면 R0, 있으면 R1 이상(`ck_judgment_confirmed_risk` · 0018 · D-273)
`hold`	보류	`hold_reason` 필수(`ck_judgment_hold_reason`) — `low_conf` · `gap2` · `cat_unknown` · `rd1` · `premise_unknown` · `law_uncovered` 여섯(0018). 응답의 보류 **종류**(`hold_kind` — 답 대기 · 고르기 대기 · 확인 못 함 · 걸린 것 없음 · 그 밖)는 계약(`app/contracts.py`)의 계산 칸이라 DB 에 칸 · 제약이 없다 (D-348 보충 36 결정 4)
`no_basis`	근거없음	위험도·유형 유지
`unjudged`	미판정(오류·타임아웃)	통과로 집계 금지(D-127)

컬럼	뜻
`subject_type` · `subject_id`	다형 참조 — 지금은 `copy_sentence` , 나중에 `claim` · `ck_judgment_subject_type`
`doc_id`	다형과 별개로 둔 FK — 문서가 지워지면 판정도 지워진다
`evidence` · `evidence_span`	근거 조문 집합(JSONB) + 근거 스팬. `evidence` 안에 위반 유형 전체 · 불가 사유 · 인코더 하한의 근거 호 (`encoder_refs`)를 싣는다
`risk_floor`	사전 하한(0~4)
`risk_encoder`	**인코더 하한**(0026) — R1 이하 · 보류 문장에만 · 근거 호 필수(0027)
`risk_final`	최종 위험도 — 두 하한 아래로 못 내린다(래칫 · 0022 · 0026). 하한 위로 올리려면 `evidence_span` 이 있어야 한다(`ck_judgment_raise_needs_evidence`)
`law_version`	법령이 개정되면 「재검증 대기」의 판단 근거
`attempt`	0-base · 0·1·2 만 가능 · `ck_judgment_attempt`

- 판정 그래프의 종착(증명서 · 지시 · 보류 · 통과)은 DB 에 저장되지 않는다. 저장되는 것은 위 네 값이다(0018 머리말).
- 다형이라 `subject_id` 의 FK 무결성은 DB 수준에서 걸리지 않는다. 조문 없는 상향은 `ck_judgment_raise_needs_evidence` 가, 근거 없는 인코더 하한은 `ck_judgment_encoder_has_basis` 가 거부한다.
- 이 PC DB 의 판정 195 행: 확정 6 · 보류 189(시험 사용분).
- 보류 종류(`hold_kind`)는 저장하지 않는다. 이력은 인코더 후보를 적지 않으므로(§ 6.3), 저장된 `low_conf` 보류만으로는 「확인 못 함」과 「걸린 것 없음」을 다시 가를 수 없다 — 저장 칸을 더할지는 남은 일이다(D-348 보충 36 결정 4).

6.3 검수 이력 저장 — 2차 이후 추가

2차 제출(09-17) 때는 판정을 DB 에 쓰는 코드가 없었다. 2026-10-13 에 `app/judgment_store.py` 가 생겼고 지금 검수 화면이 이 모듈 하나로 판정을 적는다.

항목	2026-10-21 코드
누구의 판정을 적나	**로그인한 회원만.** 비회원 판정은 응답만 내고 아무것도 쓰지 않는다(D-307)
동의	2026-10-20 변경(이서은 · 멘토 피드백 · 사용자 결정) — 「검수 이력 보관」이 선택 동의에서 **필수**로 바뀌었다. 회원이면 적고, `work_doc.consent_store` 는 늘 참 · `expires_at` 은 비운다. 종전(D-96)에는 선택 동의 ①(`consent_history_at`)이 있어야 적었다. 이 변경은 2026-10-21 현재 설계결정기록에 아직 적히지 않았다.
학습 동의	`work_doc.consent_train` 은 선택 동의 ②(`user_account.consent_improve_at`)를 그대로 옮긴다 — 학습 · 원문 조회의 유일한 문(D-128)
적는 단위	문구 하나 = `work_doc` 하나 · 엔진이 나눈 문장 하나 = `copy_sentence` 하나 + `judgment` 하나. 문서 제목에는 원문을 넣지 않는다(원문은 `copy_sentence.raw` 한 곳)
적지 않는 것	판정 대상 아님(`not_claim`) 문장 · 분기 질문의 후보(`branches`) · 인코더 추정 후보와 확률
위반 유형	`judgment.violation_type` 은 한 칸이라 첫 유형을 적고, 전부는 `evidence.violations` 에 남긴다
실패	저장이 실패해도 판정 응답은 나간다. 되돌리고(rollback) 예외 종류 이름만 로그에 남긴다 · 화면에는 「저장 못 함」 상태를 알린다
다시 판정	**지금 코드** — 품목 · 전제를 고른 재판정은 같은 사람 · 같은 문구의 최근(6시간 안) 기록을 지우고 새로 적는다(`replace_review` · 이서은 10-20). 지우면 `work_doc` CASCADE 로 문장 · 판정이 함께 지워진다. 고르지 않고 질문에 답한 재판정은 새 기록으로 쌓이고, 고른 뒤 답한 재판정은 적지 않는다(`review_answer`). **팀장 결정**(D-348 보충 36 결정 2 · 10-21) — 고른 재판정은 앞 기록을 지우지 않고 **새 판정 행**으로 쌓고 품목 출처를 `user_selected` 로 남기며, 홈 통계는 같은 사람 · 같은 문구의 가장 최근 판정만 센다. 기준 커밋 `b54246d` 에서는 이 결정이 아직 코드에 반영되지 않았다. 「대체됨」 칸 + 마이그레이션 안은 고르지 않았으므로 스키마는 바뀌지 않는다. **덧붙임** — 기준 커밋 뒤 2026-10-22 팀장 커밋 `4d8f09e` 로 반영됨 — `replace_review` 를 걷고 고른 재판정도 새 행으로 저장(고른 뒤 답한 재판정도 저장) · 홈 통계 · 반복 지적은 같은 문구의 최신 판정만 셈

제기된 이슈 — 보류가 많으면 서비스가 한 단계 더 되는 것 아닌가

구분	내용
현 진행 (2026-10-21)	판정은 네 값이고 `hold` 에는 `hold_reason` 이 필수다 — `ck_judgment_hold_reason` 이 「hold ⇔ 사유 있음」을 양방향으로 강제한다. 보류 사유는 넷에서 여섯으로 늘었다(0018 · 전제 미확인 · 법 미적용). 인코더만 위험을 잡은 보류 문장도 R1 까지 위험도를 기록할 수 있게 됐다(0026 · 근거 호 필수 0027)
지적 중간발표 09-15	문구 10개에 3 위반 / 6 보류 / 1 통과가 나오면 실용성이 크게 떨어질 수 있다. 보류분을 다시 법무팀 · 외부 심의기관에 보내야 하므로 서비스가 추가 단계가 될 수 있다. 따라서 보류 비율을 낮추는 것이 중요하다. 단순히 「보류」라고 보여주기보다 왜 보류됐는지 사용자에게 설명하는 기능도 고려할 만하다
결론	보류 사유를 스키마가 강제하는 것까지가 이 문서의 범위다. 「보류」 단독 출력은 만들지 않고 사유 코드를 항상 함께 낸다. 보류 비율 자체를 낮추는 것은 판정 코어의 과제이고 이 문서에서 약속하지 않는다 — 최신 dev 측정(원장 10-21 ⑤ · 조건부 채점 783 행)에서 판정을 내린 행은 66(coverage 8.4 %)이고 나머지는 보류다

제기된 이슈 — 결과를 확정적 판정처럼 제시하면 안 된다

구분	내용
현 진행	`confirmed` · `hold` · `no_basis` · `unjudged` 네 값과 위험도 R0~R4 를 저장한다
지적 강사 가이드 08-28(2차 제출본 기록)	결과는 법률적 의사결정을 지원하는 성격이므로, 서비스 표현에서도 법률 자문을 대체하는 확정적 판정처럼 제시하지 않는 것이 중요하다
결론	`confirmed` 는 판정 절차가 끝났다는 뜻이지 법적으로 확정됐다는 뜻이 아니다. 화면·API·보고서 어디에서도 확정 판정으로 표기하지 않는다. 「통과」는 「법령 위반 소지가 검출되지 않았다」는 뜻이지 안전 보장이 아니다 (`DISCLAIMER.md`)

7. 인증

7.1 왜 계정 테이블이 거버넌스의 일부인가

D-66 이 「decided_by·reviewed_by 를 채우려면 누가 로그인했는지 알아야 한다」고 적었다. `app_account` 가 그 「누가」다. 없으면 2인 확인의 서명은 손으로 적는 글자에 불과하다.

7.2 설계 — 계정 표가 둘이다

항목	관리자 app_account (0012)	일반 회원 user_account (0017)	DB 강제
식별	이니셜(유일) · 표시명	이메일(유일 · 소문자 · `@` 검사) · 이름 · 소속(자유 기재)	`app_account_initials_key` · `ix_user_account_email` (UNIQUE) · `ck_user_account_email_lower` · `ck_user_account_email_at`
해시	Argon2id(OWASP 권장 m=19456 · t=2 · p=1 · PHC 문자열)	같음	`ck_app_account_phc` · `ck_user_account_phc`
역할	`role = 'governor'` 만	역할 칸 없음 — 행동이 역할을 정한다(D-66)	`ck_app_account_role`
생성	`launcher.py admin-add <이니셜>` (메뉴 35) · 가입 화면 없음	가입 화면 `/u/signup` · 약관 판 · 약관/면책 동의 시각 · 선택 동의 시각 기록	—
세션	서명 쿠키 `copylane_session` (hmac · HttpOnly · SameSite=Lax)	서명 쿠키 `copylane_user`	—
CSRF · 시도 제한	이중 제출 토큰 · 시도 제한 (보안점검 P2-11)	로그인 뒤 쓰기 화면에 이중 제출 토큰	—
끄기	`disabled_at`	`disabled_at` (마이페이지 「끄기」)	—

7.3 일반 회원 테이블 — 2차 때는 없었고 0017 에서 생겼다

2차 제출본은 「일반 사용자 테이블은 아직 없다 — `/u/login` · `/u/signup` 은 입력창이 disabled」라고 적었다. 2026-09-22 결정(D-260)으로 `user_account` · `notice` · `terms` · `ticket` 이 생겼고, 가입 · 로그인 · 문의가 실제로 돈다. 같은 마이그레이션이 `work_doc.owner_id` 에 FK 를 걸었다 — FK 를 걸기 전에 값이 든 행을 세고, 있으면 멈추도록 했다(출처를 모르는 값을 지우거나 NULL 로 덮지 않는다).

만들지 않은 것 — 기업 회원 · 가입 심사 서류 · 결제 표(D-260 ④⑤⑥). 이메일 인증은 보내지 않는다(`email_verified_at` 을 채우는 코드가 없다). 조직 ID 칸(`org_id`)은 어느 표에도 없다. 개인정보 접속기록 1년 보관은 아직 표가 아니고 로거로만 남긴다(`app/auth.py`).

8. 저장소(파일시스템) 설계 — 위치가 곧 권한

디렉터리 · 파일	DB 대응	규칙	이 PC (10-21)
`data/raw/<원천>/`	(DB 미적재)	원본 무손상 보관 · 소비 금지(D-19)	원천 폴더 28개 · 일부만 있다(원문은 기기마다 받는다)
`data/g3/`	`fragment.grade = 'G3'`	원문 자유	디렉터리만 있다 (비어 있음)
`data/g2_facts/`	`fragment.grade = 'G2'`	사실만 — 원본은 추출 후 삭제	디렉터리만 있다 (비어 있음)
`data/g2_norepub/`	`source_constraint.flag = 'NOREDIST'`	재배포 금지	디렉터리만 있다 (비어 있음)
`data/quarantine/`	`fragment.grade = 'G0'`	어떤 스크립트도 읽지 않는다	디렉터리만 있다 (비어 있음)
`data/.g1_blocked/`	`fragment.excluded = true`	빈 디렉터리 — 존재 자체가 「배제했다」는 기록	있음(비어 있음)
`data/derived/`	`sentence` · `chunk` · `dict_entry` · `golden_sample` 등	파생물 — 적재기의 입력	있음
`data/manifest.jsonl`	`collect_manifest`	수집 원장 — git 으로 커밋된다	28,277 줄
`data/derived_manifest.jsonl`	—	**파생물 원장** — 어느 파일이 어떤 판(sha)인지. 원천 파일이 바뀌면 쓰기를 거부한다(사람이 `--accept-loss` 로 판정) · git 으로 커밋된다	245 항목
(업로드 경로)	`upload_blob` · NOTRAIN	경로는 `upload_blob` 에만 · 학습·색인 코드가 읽지 않는다	업로드 기능이 아직 없다(행 0 · 쓰는 코드 없음)

8.1 기기 사이의 공유 — 2차 이후 추가

무엇	어디	누가 올리고 받나	근거
파생물	팀 비공개 공유 저장소(Google Drive · `.env` `DATA_STORE`) · sha 이름	정본 기기(`DATA_ROLE=canonical` · 클론 B)만 `data-publish` 로 올린다 · 사본 기기(`replica` — 이 PC 포함)는 `data-sync` 로 받는다	`scripts/data_store.py` · D-226 · D-247
팀원이 수집한 원문	원문 받은편지함(`.env` `RAW_INBOX`)	`raw-publish` 로 올린다 · 정본 기기가 합친다	`scripts/raw_inbox.py` · D-250
팀장 기기 사이 원문	원문 거울	`raw-mirror-publish` · `raw-mirror-sync`	`scripts/raw_mirror.py` · D-256

- **추출은 정본 기기에서만** 한다(D-226) — 두 기기가 따로 추출하면 파생물 sha 가 갈린다.

- `data/**` 는 커밋하지 않는다 — 재배포 제한 자료가 섞여 있고, G2 자료는 사실을 뽑은 뒤 원본을 지우는 것이 규칙이다(D-19). 라벨 · 라벨 시트도 공개 git 에서 뺐다(D-249). 인용 광고 문구 원문이 들어 있기 때문이다. git 이 나르는 것은 원장 두 개(`data/manifest.jsonl` · `data/derived_manifest.jsonl`)와 `data/derived/golden/split_manifest.json`, 빈 디렉터리 표시(`.gitkeep`)뿐이다(`.gitignore`).

> **디렉터리와 DB 는 이중화가 아니라 이중 방어다.** 디렉터리는 사용 시점을 막고, DB 제약은 적재 시점을 막는다. 팀원이 파일을 그냥 열어도 DB 에는 못 들어간다(D-19).

---

9. 뷰 — 실수를 구조로 막는다

뷰	막는 실수	행(이 PC DB)
`v_current_chunk`	`superseded_at` 필터를 잊는 것 · 배제된 fragment 를 읽는 것	6,686
`v_publishable_golden`	재배포 금지 유래가 섞인 골든셋을 공개 배포하는 것 — `redistributable = true` 만	1,364
`v_golden_scored`	조건 M(모호) · D(판정 대상 아님) 행을 채점에 섞는 것(0024)	6,510
`v_golden_legal`	「위반 목록이 비었다 = 적법」으로 읽는 것 — 위반이 비고 조건이 L 이거나 없는 행만(0024)	78
`v_risk_lookup`	위험도를 벡터 검색으로 구하는 것 · 2인 서명 전 행을 쓰는 것(0013) · 0023 이 별표1 목 칸 다섯을 더했다	38

0009 사고 — `v_current_chunk` 를 `SELECT c.*` 로 만들었는데 뷰는 생성 시점의 열 목록으로 고정된다. 0008 이 chunk 에 열을 더하고도 뷰를 다시 만들지 않아 검색이 503 으로 죽었다. 그 뒤로 chunk 나 golden_sample 의 열을 더하거나 이름을 바꾸는 판(0010 · 0011 · 0019 · 0021 · 0024 · 0025)은 같은 파일에서 뷰를 다시 만든다(D-197).

10. 인덱스 전략

인덱스	종류	이유
`ix_chunk_current`	부분 `WHERE superseded_at IS NULL`	검색은 항상 현행만 · (law_id, article)
`ix_chunk_tsv`	GIN	어휘 검색(BM25 갈래) · 0010 에서 생성열로 추가
`ix_document_current`	부분 `WHERE superseded_at IS NULL`	현행 문서 조회
`ix_fragment_grade`	부분 `WHERE excluded = false`	배제되지 않은 추출물만
`ix_sanction_lookup`	부분 `WHERE superseded_at IS NULL`	위험도 조회의 주 경로 (violation_type, offense_count)
`ux_sanction_rule_key`	부분 유일 `WHERE rule_key IS NOT NULL`	서명된 원천의 행 열쇠는 하나뿐(0023)
`ix_golden_viol`	GIN	`violations` 배열 검색 — 유형별 집계
`ix_golden_redis` · `ix_golden_split` · `ix_golden_origin`	B-tree	공개 배포 · 분할 · 출처 필터
`ix_judgment_subject` · `ix_judgment_doc_id`	B-tree	다형 참조 조회 · 문서별 판정
`ix_work_doc_owner_id` · `ix_work_doc_session_id` · `ix_work_doc_expires_at`	B-tree	내 이력 · 세션 · 파기 대상
`ix_app_error_log_occurred` · `ix_app_error_log_level_occurred`	B-tree (시각 내림차순)	최근 오류 조회 · 보관 기한 삭제
`ix_ticket_requester_id` · `ix_ticket_expires_at`	B-tree	내 문의 · 파기 대상

인덱스 70개 = 기본키 32 · 유일 제약 4 · 유일 인덱스 3 · 일반 31(이 PC DB `pg_indexes` · `alembic_version` 포함). `app_account.initials` 에는 유일 제약 인덱스와 일반 인덱스 `ix_app_account_initials` 가 함께 있다.

> 벡터 인덱스(HNSW / IVFFlat)는 2026-10-21 에도 만들지 않았다. 청크 6,686 규모에서는 순차 스캔으로 충분하고, `ivfflat` 의 `lists` 를 잘못 잡으면 재현율이 조용히 떨어진다. 청크가 만 단위가 되면 실측해서 만든다(D-192).

11. 마이그레이션 이력

Alembic 단일 체인 · `launcher.py doctor` 가 head 일치를, `launcher.py db-drift` (메뉴 40)가 스키마 선언 ↔ 실제를 임시 DB 둘을 떠서 잰다. `scripts/db_fresh_check.py` (메뉴 39)가 빈 DB 에서 `upgrade head` 가 도는지 검사한다.

이 PC DB head = `0027_judgment_encoder_basis` (2026-10-21 조회 · 저장소 head 와 같다 · `ksr b54246d` 까지 새 마이그레이션 없음)

> 스키마를 바꿀 때 고치는 곳은 둘이다 — 거버넌스 층은 `db/schema.sql` + 새 마이그레이션, 런타임 층은 `app/models.py` + 새 마이그레이션. 런타임 층 CHECK 글자는 모델의 `CheckConstraint` 와 같아야 하고, 게이트 시험이 둘을 맞댄다(0018 · 0022 · 0026 · 0027 머리말). 되돌릴 수 없는 변경(칸 삭제)은 골든셋이 쌓인 뒤 금지다 — 열을 지우면 라벨이 사라진다.

판	날짜	층	무엇	근거
0001	09-09	데이터	거버넌스 층 — 입력이 동결본 `db/schema_0001.sql` 로 고정	D-225
`6c1f5f12e174`	09-09	런타임	런타임 층 최초 6표(`work_doc` · `copy_sentence` · `batch_run` · `judgment` · `upload_blob` · `slot_assignment`)	D-103 ~ 105
0003 ~ 0004	09-10	데이터	골든셋 라벨 · `product_fact` 유니크	
0005 ~ 0007	09-10	데이터	ENUM 후보 등재 · 널 우회 제약 차단 · 그 되돌리기	D-170
0008 ~ 0011	09-12	데이터	청크 문맥 · 뷰 재생성 · 어휘 색인(GIN) · 쪼갠 조각	D-158 · D-197 · D-193 · D-199
0012	09-12	런타임	`app_account`	D-213
0013	09-13	데이터	`v_risk_lookup` 에서 2인 확인 강제	D-66 · D-170
0014	09-13	런타임	stamp 로 건너뛴 DB 치유 — `judgment` 제약 재선언	D-221
0015	09-14	데이터	`document.annex_no`	
0016	09-22	런타임	`app_error_log` — WARNING 이상 오류 로그 · 마스킹 · 90일	D-261
0017	09-22	런타임	`user_account` · `notice` · `terms` · `ticket` · `work_doc.owner_id` FK	D-260
0018	09-23	런타임	`judgment` 보류 사유 6 · 품목 출처 칸 · 확정 행 불변식(위반 없으면 R0)	D-273 · D-276 · D-277
0019	09-24	데이터	`chunk.category` → `chunk.law` (4법 CHECK · 법 ID 로 채움 · 재임베딩 없음)	D-271
0020	09-25	데이터	`flag_t` 에 `ND` (변경금지 — 파생 데이터셋 금지)	D-110
0021	09-28	데이터	`chunk.exempt_of` — 적용 제외 목과 그 부모	D-238 개정
0022	09-29	런타임	`judgment` 최종 위험도 ≥ 하한(래칫)	D-09
0023	10-02	데이터	`sanction_rule` 행 열쇠 · 별표1 목 · 덮음 · 원문 인용 · 사실 칸 · 판 sha	D-305 · D-309 · D-310
0024	10-05	데이터	`golden_sample.cond` · `evidence_candidate` · 뷰 `v_golden_scored` · `v_golden_legal`	D-317

판	날짜	층	무엇	근거
0025	10-12	데이터	`chunk.coord_block` — 인용 좌표를 막는 이유	D-293 개정
0026	10-16	런타임	`judgment.risk_encoder` — 인코더 하한 칸 · 하한은 사전 또는 인코더 · R1 이하 · 보류 문장에만	D-348 보충 2
0027	10-16	런타임	인코더 하한에는 근거 호(`evidence.encoder_refs`)가 있어야 한다	D-348 보충 2 결정 8

굵은 판이 2차 제출(09-17, head 0015) 뒤에 더해진 12개다.

- **기존 행이 새 제약을 어길 수 있는 판은 먼저 센다.** 0006 · 0014 · 0017 · 0018 · 0019 · 0022 · 0027 은 깨는 행이 있으면 멈추고, 지우거나 고치지 않는다(각 파일의 GUARD).
- **되돌리기를 막아 둔 판** — 0003 · 0005 · 0013 · 0014 · 0018 · 0020 · 0022 · 0023 · 0024 · 0026 의 `downgrade()` 는 `NotImplementedError` 다. 되돌리면 2인 확인 · 래칫 · 조건 칸 같은 규칙이 풀리므로 사람이 판정한다.
- 0013 사고 — `ck_sanction_four_eyes` 는 둘 다 NULL 이면 통과한다. 막는 자리를 읽는 쪽(`v_risk_lookup`)으로 옮겼다 — 적재는 되고, 서명 전에는 위험도 하한이 안 잡힌다.
- 0014 사고 — 0006 이 거버넌스 마이그레이션인데 런타임 표 `judgment` 에 제약을 걸어, 새 DB 에서는 그 부분만 빠졌다. 0014 로 다시 선언하고 빈 DB 검사(메뉴 39)를 만들었다.

---

12. 실측 검증

12.1 스키마 객체가 설계대로 있다 — 2026-10-21 15:44 KST · 이 PC · SELECT 만

객체	2차(09-17 · head 0015)	최종(10-21 · head 0027)
데이터 층 표	19	19
런타임 층 표	7	**12**
뷰	3	**5**
열거형	14	14(`flag_t` 값 11 → 12)
CHECK	25(`ck_` 이름만)	**53**(전부 · `ck_` 이름만 51)
외래키	27(CASCADE 25 · SET NULL 2)	**32**(CASCADE 25 · SET NULL 2 · NO ACTION 5)
인덱스	58	**70**
벡터 칸	`vector(1024)`	`vector(1024)` (`chunk_embedding.embedding`)

2차 칸은 2차 제출본(09-17 클론 A 실측)의 값을 그대로 옮겼고, 두 값 모두 맞다. 다만 세는 기준이 다르다. 2차 조회는 CHECK 를 이름이 `ck_` 로 시작하는 것만 셌고(`conname LIKE 'ck_%'`), 인덱스는 `pg_indexes` 전체를 셌다(2차 캡처의 조회 SQL). 이번 개정은 CHECK 를 이름과 관계없이 전부 셌다 — 53 = `ck_` 51 + 이름 없는 CHECK 2(`golden_sample_unit_check` · 0003 · `golden_sample_cond_check` · 0024). 0015 시점(커밋 `16678c3`)의 CHECK 는 `ck_` 25 · 전부 26 이다. 인덱스는 0016 ~ 0027 이 12개를 더했다(0016 `app_error_log` 3 · 0017 `user_account` · `notice` · `terms` · `ticket` 8 · 0023 `ux_sanction_rule_key` 1) — 70 − 12 = 58 로 2차 값과 같다.

조회 방법 — 읽기 전용 트랜잭션에서 `pg_tables` · `pg_views` · `pg_enum` · `information_schema.columns` · `pg_constraint` · `pg_indexes` · 표마다 `count(*)` 를 읽었다. 결과 전문은 별도 파일 `DB실측_카탈로그_2026-10-21.txt` 로 함께 남겼다. 런타임 층은 `app.models` 의 메타데이터와 칸 · CHECK 이름을 맞대 차이가 0 이었다.

12.2 제약 8종이 위반을 거부했다 — 2026-09-17 실측

이번 개정에서는 쓰기 시험을 다시 돌리지 않았다(이 PC DB 는 SELECT 만 했다). 아래 거부 결과는 2차 제출 때(2026-09-17 · 클론 A · `BEGIN … ROLLBACK`) 실측이다. 10-21 에는 여덟 제약이 같은 이름과 정의로 DB 에 있는 것을 카탈로그로 확인했다.

#	제약	시도한 것 (09-17)	결과 (09-17)	10-21 정의
1	`ck_source_four_eyes`	판정자와 확인자를 같은 사람으로	거부	있음 · 같음
2	`ck_source_by_attr`	출처 표기(attribution) 없는 G3 소스	거부	있음 · 같음
3	`ck_golden_injected_not_holdout`	주입본을 test_sentence 에	거부	있음 · 같음
4	`ck_segment_k_anon`	5명짜리 세그먼트	거부	있음 · 같음
5	`ck_chunk_tokens`	900 토큰 청크	거부	있음 · 같음
6	`ck_sentence_span`	오프셋 역전(start 10 > end 3)	거부	있음 · 같음

#	제약	시도한 것 (09-17)	결과 (09-17)	10-21 정의
7	`ck_app_account_role`	role = 'user' 계정	거부	있음 · 같음
8	`ck_app_account_phc`	Argon2id 가 아닌 해시	거부	있음 · 같음

2차 뒤에 생긴 제약(0016 ~ 0027 · `user_account` · `judgment` 의 인코더 하한 등)의 거부 시험은 이 문서에서 다시 돌리지 않았다. 해당 마이그레이션의 게이트 시험이 그 자리를 맡는다(각 파일 머리말).

12.3 캐스케이드 삭제로 9개 테이블이 전부 0 이 됐다 — 2026-09-17 실측

2026-09-17 클론 A 에서 `DELETE FROM source` 한 줄로 `fragment` · `document` · `sentence` · `chunk` · `chunk_embedding`(벡터) · `dict_entry` · `sanction_rule` · `golden_sample` · `transform_pair` 가 1 → 0 이 됐다(`BEGIN … ROLLBACK`). 10-21 카탈로그에서 이 경로의 FK 20개가 모두 `ON DELETE CASCADE` 로 그대로 있음을 확인했다(§ 4.1). 이 결과가 「거버넌스를 코드로 강제했다」의 증거다 — 이용조건 제공처에서 「쓰지 마세요」 회신이 오면 한 줄로 회수된다, 벡터까지.

12.4 적재 현황 — 이 PC DB · 2026-10-21 15:44 KST

이 PC 는 정본 기기(클론 B)에서 만든 파생물을 받아 적재한 개발용 복제본이다. 판(지문)이 같으면 행 수도 같아야 하지만, 이 표는 이 PC 의 값이다.

표	행	비고
`source`	53	레지스트리 등재 70 중 적재분 · 건너뜀 17(2인 확인 미완 · attribution 없음 · 원장 10-06 ①)
`source_constraint` · `source_use`	58 · 212	
`collect_manifest`	19,122	원장 파일 28,277 줄과 다름(§ 5.1)
`fragment`	21	G3 9 · G2 12 · 배제 0
`document`	62	
`chunk` = `chunk_embedding`	6,686 = 6,686	KURE-v1 · 원장 10-13(B)와 같다
`dict_entry`	4,342	사전 `2ff0168c4a28`(원장 10-19 ①)
`product_fact`	447	
`sanction_rule`	38	2인 서명됨 — `v_risk_lookup` 38
`violation_article`	14	
`golden_sample`	8,383	골든 `94a79fe34363` · train 5,894 · test_sentence 2,489(원장 10-18 ⑭)
`sentence` · `transform_pair` · `penalty_rule` · `penal_clause` · `segment` · `dataset_manifest`	0	채우는 적재 단계가 없다
런타임 층 `work_doc` · `copy_sentence` · `judgment` · `user_account` · `ticket`	43 · 195 · 195 · 7 · 2	나머지 7표 0 · 이 PC 시험 사용분(조회 시각에 따라 늘어난다)

2차 제출(09-16 클론 B)과 비교 — `source` 32 → 53 · `chunk` 2,400 → 6,686 · `golden_sample` 1,910 → 8,383 · `dict_entry` 536 → 4,342 · `sanction_rule` 0 → 38 · `violation_article` 0 → 14 · `document` 32 → 62 · `fragment` 11 → 21.

---

13. 미확정 · 검토 중 (2026-10-21)

#	항목	상태
1	K_MIN 값(k-익명성)	현재 20 은 임시값 · CHECK 에 상수로 박혀 있어 확정 시 마이그레이션 필요. 세그먼트 기능 자체가 설계 단계(D-270 재검토 대기)
2	검수 이력 — 고른 재판정을 새 행으로 쌓기 (출처 `user_selected` · 홈 통계는 최근 판정만)	**결정됨 · 코드 반영 대기**(D-348 보충 36 결정 2 · 기준 커밋 코드는 `replace_review` 로 바꿔 적는다 · 스키마 변경 없음 · §6.3). **덧붙임** — 기준 커밋 뒤 2026-10-22 팀장 커밋 `4d8f09e` 로 반영됨 — `replace_review` 를 걷고 고른 재판정도 새 행으로 저장(고른 뒤 답한 재판정도 저장) · 홈 통계 · 반복 지적은 같은 문구의 최신 판정만 셈
3	검수 이력 보관을 필수로 바꾼 것(10-20)	코드에는 반영됨 · 설계결정기록에는 아직 적히지 않음(§6.3)
4	벡터 인덱스	청크가 만 단위가 되면 실측해서 만든다(D-192)
5	`input_token_count` 상한 CHECK	리랭커 미선정이라 걸지 않았다(D-200)
6	파티셔닝	`sentence` 가 수만 행을 넘으면 검토 · 지금은 0 행
7	집계·메타 표의 회수 처리	`segment` · `dataset_manifest` 가 캐스케이드 밖 — 회수 요청이 닿을 때 절차 미정(§4)
8	표만 있고 쓰는 코드가 없는 표	`batch_run` · `upload_blob` · `slot_assignment` · `terms` (쓰기) · `sentence` · `transform_pair` · `penalty_rule` · `penal_clause`
9	접속기록 1년 보관	표가 아니라 로거로만 남긴다(`app/auth.py`) — 실제 외부 사용자를 받기 전에 선행해야 한다(D-260 다시 볼 조건)
10	소스별 `evidence_tier` (L1~L4 근거 신뢰도)	DB 에 칸이 없다. 관리자 화면(`admin/source_detail.html`) 초안만 있고 버튼이 비활성이다. 제안 · 이력 표는 만들지 않았다 — 스키마 변경 승인 전이다(§13.1)
11	`org_id`	어느 표에도 없다(D-66 초안의 「지금 넣는다」는 집행되지 않았다)
12	`collect_manifest` ↔ 원장 파일 줄 수	19,122 ↔ 28,277 · 적재 규칙(§5.1)상 같을 필요는 없으나 원천별 내역은 세지 않았다

13.1 근거 신뢰도 등급(evidence_tier) 신설 제안 — 승인 전

관리자 화면에서 관리할 등급은 `evidence_tier` (L1~L4, 근거 신뢰도)이고, 기존 `source.grade` (G0~G3, 이용조건 게이트)는 표시만 하고 다시 판정하지 않는다. 두 축은 독립이다 — 예를 들어 국가법령정보센터 조문은 G3 + L1, 팀이 정리한 사전은 G2 + L4 다. 화면에서 G 등급을 다시 판정하면 안 되는 이유는 수집 스크립트가 실제로 그 값(`registry.require()`)을 보고 실행 여부를 정하기 때문이다.

구분	`grade` (G0~G3)	`evidence_tier` (L1~L4)
무엇	이용조건 게이트	근거 신뢰도
누가 매기나	수집 파이프라인	관리자 화면(신설 제안)
화면에서	읽기 전용	제안 → 2인 확인 → 이력

제안된 구조는 ① `source` 에 `evidence_tier` 와 2인 확인 칸을 더하고 ② 승인 대기 상태를 담는 제안 표 ③ 이력 표 (`superseded_at IS NULL` 이 현재값 · 되돌리기는 삭제가 아니라 새 행)를 두는 것이다. 검토할 점이 넷 남아 있다 — 같은 소스에

대기 제안이 여럿 들어갈 수 있음(부분 유일 인덱스 필요) · 현재값 유일성을 강제하는 제약이 없음 · `reviewed_by` 가 NULL 이면 CHECK 가 통과함(0013 과 같은 형태) · 캐시 정합성을 트리거로 지킬지 코드에 맡길지.

2026-10-21 현재 상태 — DB 에 칸 · 표가 없다. 관리자 화면 `admin/source_detail.html` 은 초안으로만 있고 버튼이 모두 비활성이다. 계정 · 운영 표를 정한 결정(D-260)도 이 제안을 범위 밖으로 두었다.

---

14. 근거

문서 · 코드	무엇인가
`db/schema.sql`	데이터 층 정본
`app/models.py`	런타임 층 정본(ORM)
`alembic/versions/*.py` · `db/migrations/*.sql`	변경 이력 0001 ~ 0027
`app/judgment_store.py`	검수 이력 저장 규칙(§6.3)
`app/auth.py` · `scripts/admin_account.py`	인증 · 관리자 계정
`scripts/load_db.py` · `scripts/data_store.py` · `scripts/raw_inbox.py` · `scripts/raw_mirror.py`	적재 · 파생물 공유 · 원문 받은편지함 · 원문 거울
`docs/02_설계/거버넌스데이터층_DDL.md` · `docs/02_설계/DB_스키마.md` · `docs/02_설계/데이터베이스_저장소_설계_2차.md`	데이터 층 DDL 전문 · 엔티티 목록 · 저장소 안의 2차 설계 문서(2026-09-28 정리 · 오한빈)
`docs/00_거버넌스_집행계약.md`	게이트가 무엇을 어떻게 막는지
`docs/psj/notes/소스등급_판정흐름_스키마초안.md`	§13.1 제안 원본(박수진 · 초안)
`docs/00_사실원장.md`	수치의 단일 출처(10-06 ① · 10-13(B) · 10-18 ⑭ · 10-19 ① · 10-20 ① · 10-21 ⑤)
이 PC DB 카탈로그 조회(2026-10-21 15:44 KST)	§1 · §4 · §5 · §6 · §9 · §10 · §12 의 수
중간발표 Q&A 정리(09-15 · 저장소 밖 팀 문서 `Final_Project_3\문서\중간발표\CopyLane 중간발표 Q&A 정리.md` — 2 · 5절)	이슈 블록의 중간발표 「지적」 칸 원문(2026-10-22 대조)
멘토링 기록(09-16) · 강사 가이드 FINAL DAY1(08-28)	이슈 블록의 「협의」 칸 · 강사 가이드 「지적」 칸 — 원 기록은 저장소에 없어 2차 제출본 인용

주요 결정 — D-09(래칫) · D-18(등급은 추출물 속성) · D-19(이중 방어) · D-20(회수 경로) · D-41(pgvector 단일 인스턴스) · D-59(불가 사유) · D-66(2인 확인·계정) · D-71(재배포 축) · D-74(측정 축) · D-82(카테고리는 판정 결과) · D-84(비파괴 정규화) · D-89(층 분리) · D-95(PostgreSQL 선택) · D-96(보관 동의) · D-103~105(런타임 층) · D-127(판정 4값) · D-128(학습 동의) · D-170(실패할 수 없는 단언) · D-192(벡터 인덱스 보류) · D-197(뷰 재생성) · D-200(토큰 상한) · D-213(Argon2id) · D-221(빈 DB 검사) · D-225(0001 동결본) · D-226(정본 기기 추출) · D-247 · D-249(파생물 공유 · 라벨 비공개) · D-260(운영표) · D-261(오류 로그) · D-271(청크 법 축) · D-273 · D-276 · D-277(판정 W3) · D-293(인용 좌표) · D-305 · D-309 · D-310(제재표) · D-307(비회원) · D-317(골든 조건) · D-348 보충 2(인코더 하한)